

Name: Irtiza Ali

Internship Domain: Data Science

Task Number: 01

Objective: Analyze a real-world dataset using Python and perform Exploratory Data Analysis to identify patterns, trends, relationships, and meaningful business insights. The task will focus on data cleaning, statistical analysis, feature analysis, visualization, and business recommendations.

Problem Statement: Store owners risk wasting money if they buy expensive items that turn out to be low quality and fail to sell. It is also hard to figure out which price ranges sell quickly and which ones do not, making it tough to choose the right stock. Lastly, it is difficult to clearly see which product types fill up most of the inventory, which makes smart planning harder.

Technologies Used: Python, Pandas, NumPy, Matplotlib & Seaborn, Google Colab.

Dataset Description: The dataset has 4,370 rows and 6 columns covering online marketplace products, their categories, and customer feedback.

Dataset Source: <https://www.kaggle.com/datasets/jedidiaholadele/supermarket-sales-dataset>.

Implementation Steps:

First, we get the environment ready by loading up the core Python libraries like Pandas, Matplotlib, and Seaborn so we have everything we need for data crunching and plotting.

Next, we load the `sales_raw_data.csv` file into a DataFrame and take a quick look around—checking the shape, data types, and scanning for any missing values or weird anomalies.

Then comes the data cleaning phase, where we fix up messy formats, strip out junk text, and make sure columns like price and ratings are actually readable as numbers.

After that, we run some basic statistical summaries (like checking the quartiles, which showed us that Q3 sits under \$40) to get a feel for the dataset's baseline numbers.

With clean data ready, we generate the 12 key visualizations.

Data Cleaning Process:

Missing entries in numerical features are filled using the median because the median is more robust and prevents outlier prices or ratings from distorting our statistics.

The dataset was checked for duplicate records and no duplicates were found.

The Price column originally loaded as text strings because of the currency symbol \$. We converted the column type to decimal float to allow numerical math and correlation.

We scanned numerical ranges to check for anomalies like negative prices impossible ratings outside 1-5 or negative rating counts confirming that all rows fall within realistic limits.

Feature Analysis:

Since the dataset doesn't include actual units sold, review count is the closest available signal for how much a product sells and how much attention it gets. High counts, like Amazon Basics batteries at 441,836 reviews, point to high-velocity items; low counts point to slower-moving inventory.

Price affects who buys a product and how often. 75% of catalog items are priced under \$40.00. Looking at price alongside review volume shows how cost relates to demand.

These fields let you compare how categories perform against each other where customer attention concentrates (Electronics and Home & Kitchen have millions of total reviews) and where the catalog is thin relative to demand, like children's apparel.

Rating has only a weak correlation with price or review count, but it's still a useful baseline for product reliability. The overall average across the dataset is 4.50 out of 5.0.

Correlation Analysis:

The numerical features show no strong linear relationships. Price has a near-zero correlation with both ratings (-0.036) and review counts (-0.078) proving that expensive items do not automatically receive better ratings or more customer engagement.

12+ Visualizations:

Price Distribution Histogram: This shows the frequency distribution KDE curve of the product prices, highlighting the heavy skew toward items under \$40.

Average Price Bar Chart: Compares mean product prices across leading categories to show where higher-ticket items reside.

Category Count Plot: Displays product inventory volume per category, revealing concentrations in Women's Fashion, Electronics, and Automotive.

Category Share Pie Chart: Illustrates the percentage distribution share across the top 6 product categories.

Price vs Rating Count Scatter Plot: Examines the relationship between item pricing and customer review volumes.

Rolling Average Price Line Chart: Tracks rolling average price trends across sorted catalog items.

Price Outlier Box Plot: Identifies significant high-end price outliers and quartile boundaries using the IQR method.

Category Price Violin Plot: Contrasts price density and distribution shapes across the top 5 product categories.

Correlation Matrix Heatmap: Detailed annotated correlation matrix displaying numerical variable relationships.

Multi-Variable Pair Plot: Displays pairwise scatter plots and univariate KDE curves across Price, Rating, and Rating_count to examine multi-variable distributions and correlations simultaneously.

Rating Density Distribution Plot: Displays the KDE density curve for product star ratings, emphasizing the high baseline around 4.5.

Cumulative Price Area Chart: Shows cumulative price growth across sorted catalog items.

15 Business Insights:

Low-to-mid price concentration: 75% of products are priced under \$40, the median is \$22.97, and the mean is \$48.83. Most of the catalog is inexpensive items.

Category volume: Fashion and Electronics lead: Women's Fashion 469 items, Electronics 385 items, and Automotive 375 items are the largest categories by item count.

Review volume: Electronics and Home & Kitchen lead: Consumer Electronics 7,993,501 total reviews and Home & Kitchen 7,241,511 total reviews have the highest total review counts, far ahead of smaller categories like Luggage 443,884.

Price and rating aren't correlated: Price and rating have almost no correlation. Higher prices don't mean better ratings.

Price vs. review count: Price and review count have a weak negative correlation. Cheaper items tend to have more reviews than expensive ones.

Rating vs. review count: Rating and review count have a weak positive correlation. Items with more reviews tend to rate slightly higher, but the relationship is too weak to draw firm conclusions about why.

Price outliers: Using the IQR method, 533 products are price outliers. The highest-priced are PC components and gaming laptops, including the EVGA GeForce RTX 3090 \$1,799 and the Optoma 4K Projector \$1,399.

Ratings are high overall: The overall average rating is 4.50 out of 5.0, and 66.5% of products are rated 4.5 or higher.

Smallest categories: Luggage 106 items, Health & Household 133 items, and Beauty & Personal Care 212 items have both the fewest items and the lowest total review counts.

Everyday items get the most reviews: The most-reviewed products are everyday goods: Amazon Basics batteries 441,836 reviews and Mellanni bed sheets 296,178 reviews top the list. Frequently repurchased household items tend to rack up high review counts.

Computers vs. Arts & Crafts pricing: Computers has the highest average price \$169.83, far above Arts & Crafts \$18.71. Tech products simply cost more than hobby consumables.

Mid-priced items with high engagement: A few items land in a middle ground between cheap, high-volume goods and expensive, low-volume ones: the TOZO T10 wireless earbuds 292,103 reviews at \$25.99 and the Revlon Volumizer 289,023 reviews at \$34.18 both combine strong review counts with moderate prices.

Missing data: Missing values were found in Category 25 records, Price 46 records, and Rating 30 records, likely from upstream data entry or ingestion issues.

Zero-price listings: Some products are listed at \$0 likely promotional giveaways, incomplete listings, or data entry errors. Worth reviewing before they affect revenue reporting.

Kids apparel: high engagement, few listings: Boys Fashion 650,013 reviews and Girls Fashion 863,593 reviews get a lot of engagement relative to how many items are listed in those categories. That could mean room to expand kids apparel offerings.

5 Business Recommendations:

Lean into low-price inventory: 75% of products are priced under \$40, and price has a weak negative correlation with review volume. Cheaper, everyday items already drive most of the engagement, so marketing spend is probably better aimed there than at slower-moving, higher-priced inventory.

Expand kids apparel: Boys Fashion 650,013 reviews and Girls Fashion 863,593 reviews get a lot of engagement relative to how few items are listed, while categories like Luggage 106 items, 443,884 reviews get much less. That gap suggests room to grow the kids apparel catalog.

Fix missing data and \$0 listings: Data audits found missing values in Category (25 records), Price 46 records, and Rating 30 records, plus some listings priced at \$0. Left uncorrected, these could throw off revenue reporting and anything built on top of this data, like a recommendation engine, so it's worth validating the pipeline.

Price doesn't need to track quality: Price and rating are barely correlated, so raising prices isn't likely to lift ratings on its own. The average rating is already high across the board 4.50, regardless of price tier.

Bundle around Electronics and Home & Kitchen: These two categories get far more engagement than others 7,993,501 and 7,241,511 total reviews, driven partly by high-repeat items like batteries and bedding. Bundling accessories with products from these categories could be a reasonable way to increase what customers buy alongside them.

Challenges Faced: Cleaning up the raw data and handling missing values took some trial and error, especially dealing with price outliers that stretched out our initial charts. The biggest surprise was finding out that expensive products don't actually get better ratings, which completely flipped our first assumption. It also took some effort to translate all those technical charts into practical, easy-to-understand advice for store owners.

Brief explanation of code:

- 1) Data Import: Imports pandas and loads the raw CSV data into a DataFrame.
- 2) Data Inspection: Previews rows, verifies dimensions, checks data types, and generates basic stats.
- 3) Data Cleaning: Fixes missing values, converts price strings to floats, imputes medians, and validates data ranges.
- 4) Feature Analysis: Computes IQR to flag price outliers and identifies top product categories.
- 5) Correlation Analysis: Calculates the correlation matrix and generates a heatmap using seaborn.
- 6) Data Visualization: Plots price distribution histograms and category average price bar charts.

Code Screenshots is in last.

Conclusion: This analysis cleaned and structured 4,370 e-commerce records, handling missing values and converting price fields from currency strings to numbers, then examined how products are distributed across categories, how they are priced, and what drives customer engagement.

GitHub Repository Link:

<https://github.com/irtizaali283/E-commerce-market-insights.git>

Screenshots:

Import

```
(3) import pandas as pd
```

```
(4) df = pd.read_csv("/content/sales_raw_data.csv")
```

Understanding

```
(5) df.head()
```



	Category	Sub Category	Product	Price	Rating	Rating_Count
0	Arts & Crafts	Crafting	Command Large Picture Hanging Strips, White, H...	\$13.27	4.6	103192
1	Arts & Crafts	Crafting	Scissors, iBayam 8" Multipurpose Scissors Bulk...	\$8.79	4.8	51747
2	Arts & Crafts	Crafting	Luna Bean Keepsake Hands Casting Kit DIY Pla...	\$34.99	4.6	27988
3	Arts & Crafts	Crafting	Amazon Basics Clear Thermal Laminating Plastic...	\$16.16	4.8	45653
4	Arts & Crafts	Crafting	Scotch Thermal Laminating Pouches, 100-Pack, 8...	\$15.18	4.9	54194

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
(6) df.tail()
```



	Category	Sub Category	Product	Price	Rating	Rating_Count
4365	Luggage	Boys Fashion	LONG VACATION Luggage 3 Piece Set Suitcase ABS...	\$169.99	4.5	234
4366	Luggage	Boys Fashion	LONG VACATION Luggage 3 Piece Set ABS Hardshel...	\$169.99	4.6	81
4367	Luggage	Boys Fashion	SHOWKOO Luggage Expandable Clearance Suitcases...	\$189.99	4.4	1686
4368	Luggage	Boys Fashion	Steve Madden Designer 20 Inch Carry On Luggage...	\$155.99	4.6	921
4369	Luggage	Boys Fashion	COOLIFE Luggage Expandable Suitcase PC+ABS 3 P...	\$179.99	4.7	670

```
(7) df.shape
```



```
(4370, 6)
```

```
(8) list(df.columns)
```



```
['Category', 'Sub Category', 'Product', 'Price', 'Rating', 'Rating_Count']
```

```
(9) df.dtypes
```



```
Category    object
Sub Category object
Product     object
Price       object
Rating      float64
Rating_Count int64
```

```
dtype: object
```

```
[10]
✓ On numerical_cols = df.select_dtypes(include=["int64", "float64"]).columns.tolist()
display(numerical_cols)
```

```
['Rating', 'Rating_Count']
```

```
[11]
✓ On categorical_cols = df.select_dtypes(include=["object"]).columns.tolist()
display(categorical_cols)
```

```
['Category', 'Sub_Category', 'Product', 'Price']
```

```
[12]
✓ On df.describe()
```

	Rating	Rating_Count
count	4340.000000	4370.000000
mean	4.504217	11291.828804
std	0.318628	24978.930805
min	1.000000	1.000000
25%	4.400000	292.250000
50%	4.600000	2453.500000
75%	4.700000	11195.250000
max	5.000000	441836.000000

Cleaning

```
[13]
✓ On df.isnull().sum()
```

```
0
Category    25
Sub_Category  0
Product      0
Price        48
Rating       30
Rating_Count  0

dtype: int64
```

```
[14]
✓ On df["Price"] = df["Price"].astype(str).str.replace("$", "", regex = False).str.replace(",", "", regex=False).astype(float)
df["Price"].dtype
```

```
dtype('float64')
```

```
15) median_price = df["Price"].median()
df["Price"].fillna(median_price)

***
      Price
0    13.27
1     6.79
2    34.99
3    16.16
4    15.18
...     ...
4365  169.99
4366  169.99
4367  169.99
4368  155.99
4369  179.99
4370 rows x 1 columns

dtype: float64
```

```
16) median_rating = df["Rating"].median()
df["Rating"].fillna(median_rating)

      Rating
0     4.6
1     4.8
2     4.6
3     4.8
4     4.9
...     ...
4365    4.5
4366    4.6
4367    4.4
4368    4.6
4369    4.7
4370 rows x 1 columns

dtype: float64
```

```
17) subcat_cat = df.dropna(subset=["Category"]).groupby("Sub Category")["Category"].agg(lambda x: x.mode()[0]).to_dict()
    df["Category"].fillna(df["Sub Category"].map(subcat_cat))
```

```
...
      Category
0  Arts & Crafts
1  Arts & Crafts
2  Arts & Crafts
3  Arts & Crafts
4  Arts & Crafts
...
4365  Luggage
4366  Luggage
4367  Luggage
4368  Luggage
4369  Luggage
4370 rows x 1 columns

dtype: object
```

```
18) df.isnull().sum()
```

```
0
Category    25
Sub Category  0
Product      0
Price       46
Rating      30
Rating_Count  0

dtype: int64
```

```
19) df.duplicated().sum()
```

```
np.int64(0)
```

```
20) invalid_price = (df["Price"] < 0).sum()
invalid_rating = ((df["Rating"] < 1) | (df["Rating"] > 5)).sum()
invalid_ratingcount = (df["Rating_Count"] < 0).sum()

print(f"{invalid_price},{invalid_rating},{invalid_ratingcount}")
```

[21]
✓ Oa

df.describe()

	Price	Rating	Rating_Count
count	4324.000000	4340.000000	4370.000000
mean	48.833679	4.504217	11291.828804
std	99.768467	0.318628	24978.930805
min	0.000000	1.000000	1.000000
25%	13.970000	4.400000	292.250000
50%	22.965000	4.600000	2453.500000
75%	39.990000	4.700000	11195.250000
max	1799.010000	5.000000	441836.000000

Feature Analysis

[22]
✓ Oa

df[["Price", "Rating", "Rating_Count"]].describe()

	Price	Rating	Rating_Count
count	4324.000000	4340.000000	4370.000000
mean	48.833679	4.504217	11291.828804
std	99.768467	0.318628	24978.930805
min	0.000000	1.000000	1.000000
25%	13.970000	4.400000	292.250000
50%	22.965000	4.600000	2453.500000
75%	39.990000	4.700000	11195.250000
max	1799.010000	5.000000	441836.000000

[23]
✓ Oa

```
Q1 = df["Price"].quantile(0.25)
Q3 = df["Price"].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
```

[24]
✓ Oa

```
price_outliers = df[(df["Price"] < lower_bound) | (df["Price"] > upper_bound)]
print(f"Price IQR Bounds: Lower = {lower_bound:.2f}, Upper = {upper_bound:.2f}")
print(f"Number of Price Outliers: {len(price_outliers)}")
```

Price IQR Bounds: Lower = -25.86, Upper = 79.82
Number of Price Outliers: 536

```
[25] df["Category"].value_counts().head(5)
```

Category		count
Women's Fashion		489
Electronics		385
Automotive		375
Arts & Crafts		384
Baby		380

dtype: int64

```
[26] df["Sub_Category"].value_counts().head(5)
```

Sub_Category		count
Clothing		233
Shoes		197
Watches		188
Accessories		180
Jewelry		155

dtype: int64

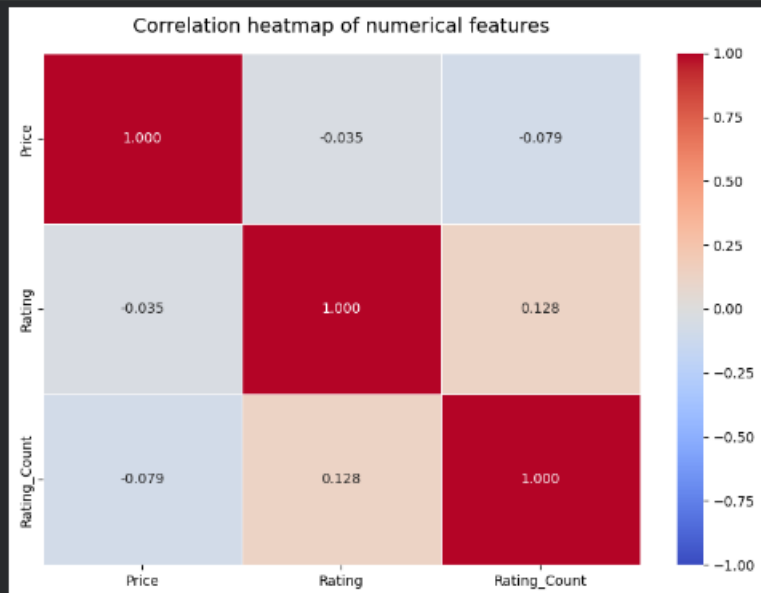
Coorelation Analysis

```
[27] import seaborn as sns
import matplotlib.pyplot as plt
```

```
[28] numerical_df = df[["Price", "Rating", "Rating_Count"]]
print(numerical_df.corr())
```

	Price	Rating	Rating_Count
Price	1.000000	-0.035302	-0.078973
Rating	-0.035302	1.000000	0.127914
Rating_Count	-0.078973	0.127914	1.000000

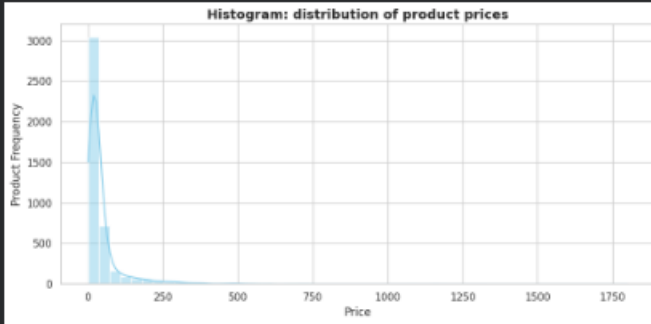
```
plt.figure(figsize=(8, 6))
sns.heatmap(numerical_df.corr(), annot=True, cmap="coolwarm", fmt=".3f", linewidths=0.5, vmin=-1, vmax=1)
plt.title("Correlation heatmap of numerical features", fontsize=14, pad=15)
plt.tight_layout()
plt.show()
```



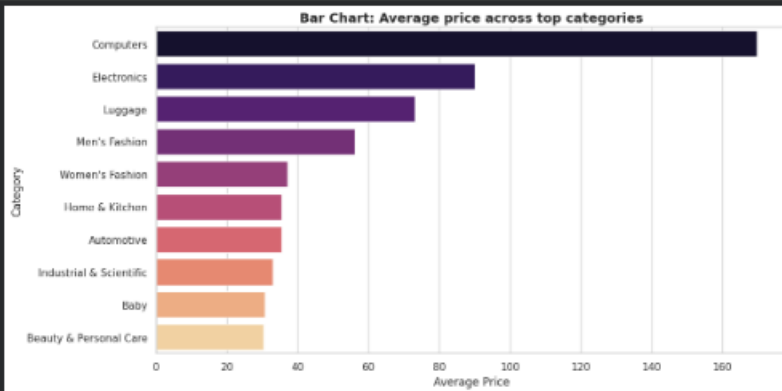
Data Visualization

```
ana.set_theme(style="whitegrid")

plt.figure(figsize=(10, 5))
ana.histplot(df["Price"], bins=50, color="skyblue", kde=True)
plt.title("Histogram: distribution of product prices", fontsize=14, fontweight="bold")
plt.xlabel("Price", fontsize=12)
plt.ylabel("Product Frequency", fontsize=12)
plt.tight_layout()
plt.show()
```



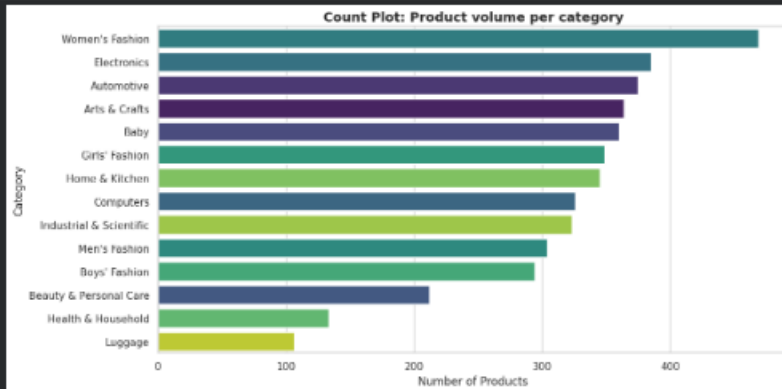
```
plt.figure(figsize=(12, 6))
avg_price_cat = df.groupby("Category")["Price"].mean().sort_values(ascending=False).head(10)
ana.barplot(avg_price_cat.values, yavg_price_cat.index, palette="magma", hue=avg_price_cat.index, legend=False)
plt.title("Bar Chart: Average price across top categories", fontsize=14, fontweight="bold")
plt.xlabel("Average Price", fontsize=12)
plt.ylabel("Category", fontsize=12)
plt.tight_layout()
plt.show()
```



```

plt.figure(figsize=(12, 8))
sns.countplot(data=df, y='Category', order=df['Category'].value_counts().index, palette='viridis', hue='Category', legend=False)
plt.title("Count Plot: Product volume per category", fontsize=14, fontweight='bold')
plt.xlabel("Number of Products", fontsize=12)
plt.ylabel("Category", fontsize=12)
plt.tight_layout()
plt.show()

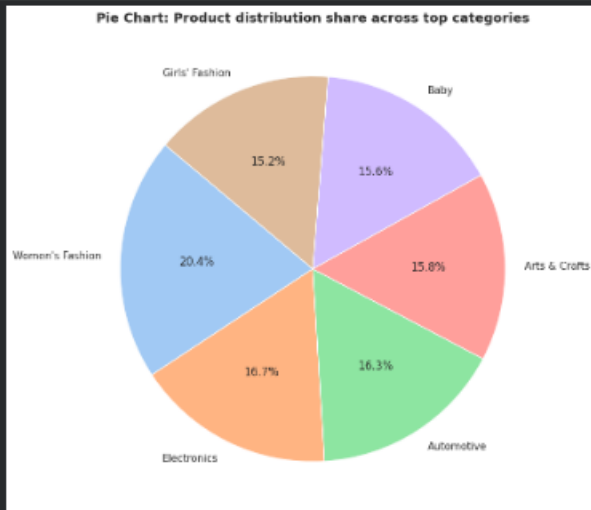
```



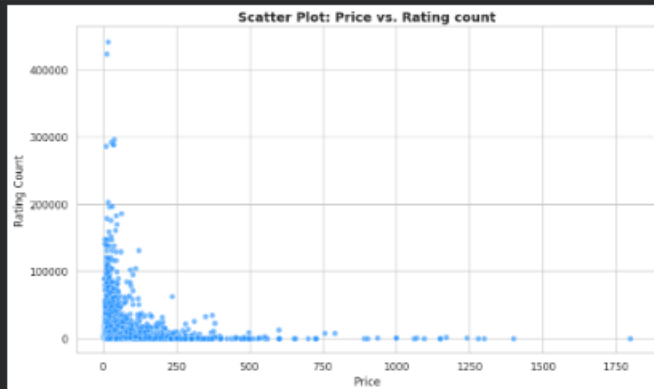
```

plt.figure(figsize=(9, 9))
cat_counts = df['Category'].value_counts().head(6)
plt.pie(cat_counts.values, labels=cat_counts.index, autopct='%1.1f%%', startangle=140, colors=sns.color_palette("pastel"))
plt.title("Pie Chart: Product distribution share across top categories", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

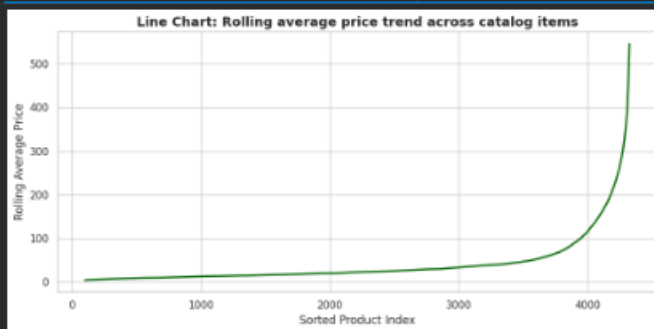
```



```
[14]: plt.figure(figsize=(10, 6))
      sns.scatterplot(data=df, x="Price", y="Rating_Count", alpha=0.6, color='dodgerblue')
      plt.title("Scatter Plot: Price vs. Rating count", fontsize=14, fontweight='bold')
      plt.xlabel("Price", fontsize=12)
      plt.ylabel("Rating_Count", fontsize=12)
      plt.tight_layout()
      plt.show()
```



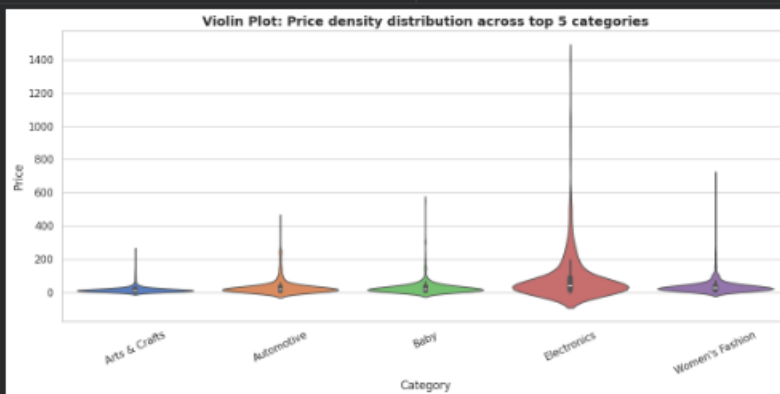
```
[15]: plt.figure(figsize=(10, 5))
      sorted_df = df.sort_values(by="Price").reset_index(drop=True)
      sorted_df["Rolling_Avg_Price"] = sorted_df["Price"].rolling(window=100).mean()
      sns.lineplot(data=sorted_df, x=sorted_df.index, y="Rolling_Avg_Price", color='darkgreen', linewidth=2)
      plt.title("Line Chart: Rolling average price trend across catalog items", fontsize=14, fontweight='bold')
      plt.xlabel("Sorted Product Index", fontsize=12)
      plt.ylabel("Rolling Average Price", fontsize=12)
      plt.tight_layout()
      plt.show()
```



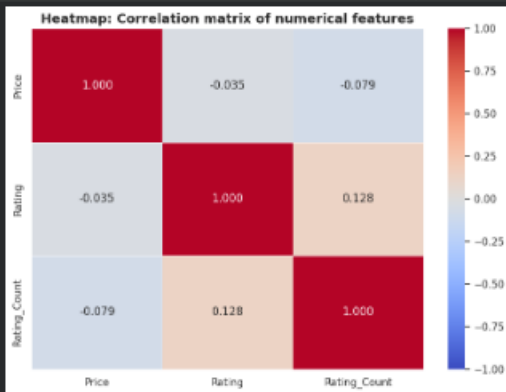
```
plt.figure(figsize=(8, 6))
sns.boxplot(x=df["Price"], color="lightcoral")
plt.title("Box Plot: Price spread and outlier identification", fontsize=14, fontweight="bold")
plt.xlabel("Price", fontsize=12)
plt.tight_layout()
plt.show()
```



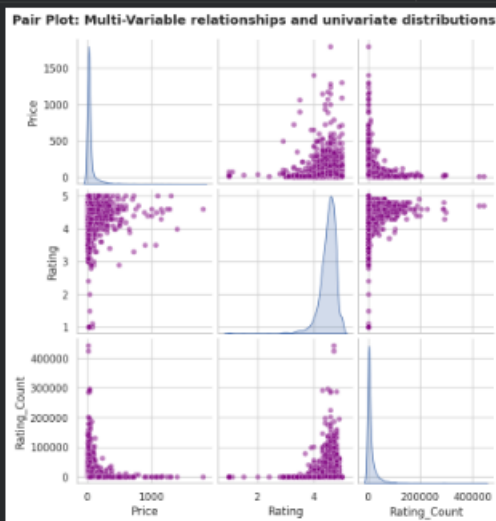
```
plt.figure(figsize=(12, 6))
top_cat_list = df["Category"].value_counts().head(5).index
filtered_df = df[df["Category"].isin(top_cat_list)]
sns.violinplot(data=filtered_df, x="Category", y="Price", palette="muted", hue="Category", legend=False)
plt.title("Violin Plot: Price density distribution across top 5 categories", fontsize=14, fontweight="bold")
plt.xlabel("Category", fontsize=12)
plt.ylabel("Price", fontsize=12)
plt.xticks(rotation=25)
plt.tight_layout()
plt.show()
```



```
plt.figure(figsize=(8, 6))
corr_matrix = df[["Price", "Rating", "Rating_Count"]].corr()
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".3f", linewidth=0.5, vmin=-1, vmax=1)
plt.title("Heatmap: Correlation matrix of numerical features", fontsize=14, fontweight="bold")
plt.tight_layout()
plt.show()
```



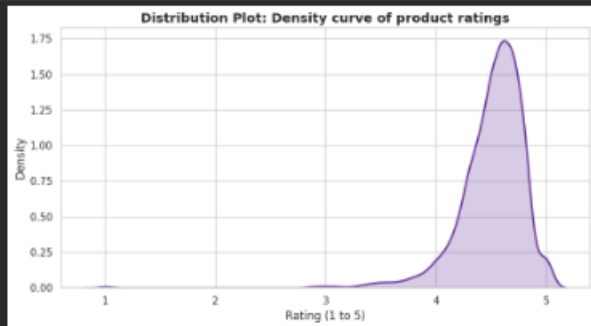
```
pairplot_fig = sns.pairplot(df[["Price", "Rating", "Rating_Count"]], diag_kind="kde", plot_kws={"alpha": 0.5, "color": "purple"})
pairplot_fig.fig.suptitle("Pair Plot: Multi-Variable relationships and univariate distributions", y1=1.03, fontsize=14, fontweight="bold")
plt.show()
```



```

plt.figure(figsize=(9, 5))
sns.kdeplot(df['Rating'], fill=True, color='rebeccapurple', linewidth=2)
plt.title("Distribution Plot: Density curve of product ratings", fontsize=14, fontweight="bold")
plt.xlabel("Rating (1 to 5)", fontsize=12)
plt.ylabel("Density", fontsize=12)
plt.tight_layout()
plt.show()

```



```

plt.figure(figsize=(10, 5))
cumulative_price = sorted_df['Price'].cumsum()
plt.fill_between(range(len(cumulative_price)), cumulative_price, color='teal', alpha=0.4)
plt.plot(cumulative_price, color='darkcyan', linewidth=1.5)
plt.title("Area Chart: Cumulative price growth across catalog", fontsize=14, fontweight="bold")
plt.xlabel("Product Index", fontsize=12)
plt.ylabel("Cumulative Price Sum", fontsize=12)
plt.tight_layout()
plt.show()

```

